



AURORA ROBOTICS

Computer Vision for Autonomous Vehicles

A Class Session Script

Capstone — Monocular Distance Estimation from First Principles

How to use this script

- ▶ This is the talking script for one ~ 90-minute session.
- ▶ Story arc: **four attempts**, three failures, one win.
- ▶ Every equation is followed by a plain-English reading.
- ▶ *Live Demo* cues tell you when to run a script.



Bakel Bakel Begededum
CTO & Chief Instructor, Aurora Robotics

Contents

What we will do today	3
1 Attempt 1 — “Just Use the Best AI Model” (<code>detect.py</code>)	3
1.1 What <code>detect.py</code> does	3
1.2 The landscape of object detection models	3
1.3 How YOLO turns pixels into boxes	5
1.4 Now let’s see if it works on our robot	6
2 Attempt 2 — “Find the Edges” (<code>edge_detect.py</code>)	6
2.1 What <code>edge_detect.py</code> does	6
2.2 The mathematics of finding an edge	7
2.3 Does this solve our problem?	7
3 Attempt 3 — “Find What’s New” (<code>robot_outline.py</code>)	8
3.1 What <code>robot_outline.py</code> does	8
3.2 The mathematics of background subtraction	8
4 From an Outline to a Number	10
4.1 Why distance and area must be related — the pinhole argument	10
4.2 Measuring k — the calibration phase	11
4.3 Why we fit several models, not just one	12
5 Attempt 4 — “Predict the Distance Live” (<code>robot_distance.py</code>)	12
5.1 Smoothing the area before computing distance	12
5.2 The extrapolation warning	13
6 What we learned today	13
Summary of Equations	14

What we will do today

We have a camera. Just a camera. No second camera, no LiDAR, no ultrasonic sensor, no depth camera. And we want to know how far away the robot is.

This sounds impossible. It is not. By the end of the class, we will have a working program that shows the distance to the robot, in centimetres, on screen, live, with a clean green outline around it.

To get there we will make four attempts. Three of them will fail in instructive ways. The fourth will work. The point of today is not the final program; it is the journey, because each failure teaches you something you cannot learn from a working answer.

1 Attempt 1 — “Just Use the Best AI Model” (`detect.py`)

Our first instinct, in 2026, is to throw a large pre-trained neural network at the problem. This is reasonable. Models like YOLO are genuinely incredible — they were trained on millions of labelled images at enormous compute cost, and they will run, real-time, on the laptop in front of you.

1.1 What `detect.py` does

The script loads a pre-trained network called YOLOv8n, opens the USB camera, and for every frame draws a coloured rectangle around every object it recognises, with a label like `person` or `chair` and a confidence score. That is the whole program in about 40 lines of Python.

Live Demo

Run `detect.py`. Point the camera at yourself, at a chair, at a coffee cup. Watch the labels appear and the boxes track you.

1.2 The landscape of object detection models

YOLOv8 is one of dozens of object detection models. Here is the field laid out so a student knows where everything sits.

Model	Year	Architecture	Vocabulary	When to use it
<i>Two-stage CNN detectors</i>				
Faster R-CNN	2015	CNN + region proposals	COCO (80)	Higher accuracy when speed is not critical. Standard baseline.
Mask R-CNN	2017	Faster R-CNN + mask head	COCO (80)	When you need pixel-level segmentation, not just boxes.
<i>One-stage CNN detectors (the YOLO family and friends)</i>				
SSD	2016	Single-shot multi-box	COCO/VOC	Fast, mobile devices, simple training.
YOLOv3	2018	Anchor-based CNN	COCO (80)	Older but well-understood baseline.

Model	Year	Architecture	Vocabulary	When to use it
RetinaNet	2017	CNN + focal loss	COCO (80)	Many small dense objects (drones, surveillance).
EfficientDet	2020	Compound-scaled CNN	COCO (80)	Edge devices, balanced accuracy/speed.
YOLOv5	2020	Anchor-based CNN	COCO (80)	The practical default for years; easy to fine-tune.
YOLOv8	2023	Anchor-free CNN	COCO (80)	What our script uses. Modern, fast, well supported.
YOLOv11 / v12	2024–25	CNN + attention	COCO (80)	Latest in the YOLO line; minor accuracy gains.
<i>Transformer-based detectors</i>				
DETR	2020	End-to-end transformer	COCO (80)	Cleanest formulation; slower to train.
RT-DETR	2023	Real-time DETR	COCO (80)	Transformer accuracy at YOLO speed.
RF-DETR	2025	DINOv2 backbone + DETR	COCO (80)	Current state-of-the-art on COCO; robust to domain shift.
<i>Open-vocabulary detectors (text-prompted)</i>				
OWL-ViT	2022	ViT + CLIP-style text	Open	Earliest practical open-vocab; slower.
Grounding DINO	2023	Transformer + grounded pre-train	Open	Highest accuracy on unusual objects.
YOLO-World	2024	YOLO + text embeddings	Open	Fast open-vocab; runs near real-time.
<i>Promptable segmentation (click or box, get a mask)</i>				
SAM	2023	Vision transformer	Anything	Click a pixel, get a clean mask of whatever you clicked.
SAM 2	2024	SAM + video	Anything	Track and segment objects through a video.

Key Idea

The first three groups share a hidden limit: they output one of a fixed list of class labels. YOLOv8 was trained on COCO, so its vocabulary is the 80 COCO classes — person, bicycle, car, motorcycle, airplane, ..., toothbrush. If your object is not on that list, the

network has no neuron that can fire for it. Group four (open-vocabulary) removes this limit but at a speed cost. Group five (segmentation) gives pixel masks instead of boxes.

1.3 How YOLO turns pixels into boxes

Now the part students always ask about: *how does it actually work?* Walk through the input–output transformation.

Input. A colour image, treated as a tensor

$$\mathbf{I} \in \mathbb{R}^{H \times W \times 3},$$

typically rescaled so values lie in $[0, 1]$. Inside the network, the image passes through three parts.

(1) **Backbone.** A stack of convolutional layers, each of the form

$$F^{(\ell)} = \text{ReLU}(W^{(\ell)} * F^{(\ell-1)} + b^{(\ell)}),$$

where $W^{(\ell)}$ is a bank of learned $k \times k$ filters and $*$ denotes convolution. After several layers, the input image has been reduced to a small spatial grid of *feature vectors*, each of which summarises a region of the original image.

(2) **Neck.** A feature pyramid that combines features from different depths so the model can detect small objects (with shallow, high-resolution features) and large ones (with deep, low-resolution features).

(3) **Head.** For each cell of the output grid at each scale, the network predicts a fixed-length vector

YOLO per-grid-cell prediction

$$(t_x, t_y, t_w, t_h, p_{\text{obj}}, p_{c_1}, p_{c_2}, \dots, p_{c_{80}}).$$

The first four numbers describe where the predicted bounding box is and how big it is. p_{obj} is the probability that there is *any* object centred in this grid cell. The 80 class probabilities say which class it is, conditional on there being an object.

Decoding the box from the raw network output:

$$b_x = \sigma(t_x) + c_x, \quad b_y = \sigma(t_y) + c_y, \quad b_w = \exp(t_w), \quad b_h = \exp(t_h),$$

where (c_x, c_y) is the grid cell location. (YOLOv8 uses an anchor-free variant with a distribution focal loss, but the high-level shape is the same.)

(4) **Non-maximum suppression.** The grid produces thousands of overlapping candidate boxes. We keep only the locally-best ones using intersection-over-union,

Intersection-over-Union (IoU)

$$\text{IoU}(B_1, B_2) = \frac{|B_1 \cap B_2|}{|B_1 \cup B_2|}.$$

Sort boxes by confidence; for each box from high to low, suppress any later box with $\text{IoU} > 0.5$ to it.

Read this as

The network looks at the image like a grid of squares laid over it. Each square raises its hand and says “I think there’s an object centred in me; it is this big, of this class, and I am this confident.” Out of thousands of raised hands, we listen only to the most confident,

and when two confident hands describe the same object, we keep one.

1.4 Now let's see if it works on our robot

Live Demo

Run `detect.py` again. This time point the camera at the ROSMASTER robot.

What you will see: chairs are labelled. People are labelled. Cups, keyboards, books are labelled. The robot is *not*, or if anything is drawn around it, the label is wrong — “microwave,” “toaster,” something that vaguely fits a small box shape.

Why this fails

The robot is not a COCO class. The neural network has 80 output neurons, one per class it was trained on. There is no neuron for “ROSMaster A1,” so no matter how clearly the camera sees the robot, the model has no way to say “robot.”

This is not a bug in YOLOv8. It is a fundamental limit of any pre-trained model: *it can only output labels it was trained on.*

The three real ways around this:

- **Custom training.** Collect ~ 200 photos of the robot, label them, fine-tune YOLOv8. Takes an afternoon. Works.
- **Open-vocabulary models** (YOLO-World, Grounding DINO). Prompt with text like “small four-wheeled robot.” Slower.
- **Fiducial markers** (ArUco, AprilTag). Stick a coded tag on the robot. 100% reliable. What real robotics labs do.

But today we are doing none of those. We are going to ask: *can we solve this without identifying the robot at all?* Maybe we don't need a class label. Maybe we just need to find a shape.

2 Attempt 2 — “Find the Edges” (`edge_detect.py`)

We don't actually care what the robot *is*. We care *where* it is. And every object, regardless of category, has a boundary — a place where its brightness suddenly differs from the background. So let us find those boundaries directly. This is called *edge detection*, and it is one of the oldest ideas in computer vision.

2.1 What `edge_detect.py` does

The script reads the camera, converts each frame to grayscale, blurs it to suppress noise, and applies the *Canny edge detector* to produce a black image with white lines along every sharp brightness change.

Live Demo

Run `edge_detect.py`. Set Show Mode to 2 (Canny). Wave your hand. The outline of your hand appears in white against black.

2.2 The mathematics of finding an edge

An edge in an image is, roughly, a place where the brightness changes sharply. If we treat the image as a function $I(x, y)$, then “brightness changes sharply” is the same as “the gradient is large.” Recall from calculus that the gradient of a 2D function is the vector of partial derivatives:

Image gradient

$$\nabla I = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right).$$

We approximate these partial derivatives in a discrete image using small 3×3 filters called Sobel kernels:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

Slide each kernel across the image; at every pixel compute the weighted sum of the 3×3 neighbourhood. The result is two images, I_x and I_y , the horizontal and vertical brightness derivatives.

Gradient magnitude

$$M(x, y) = \sqrt{I_x(x, y)^2 + I_y(x, y)^2}.$$

A high M means the brightness is changing fast at that pixel — probably an edge.

The Canny detector adds three extra steps to make the output clean:

1. **Blur first.** If we don’t blur, every speck of sensor noise produces a sharp gradient and looks like an edge. We convolve with a Gaussian first.
2. **Non-maximum suppression.** The gradient is high not just on the edge but on a thick band around it. We thin to one pixel by keeping only the local maximum along the gradient direction.
3. **Hysteresis thresholding.** Use two thresholds. Pixels above the high one are definitely edges; pixels above the low one are edges only if connected to a definite edge. This prevents weak edges from being either fully accepted (creating noise) or fully rejected (creating gaps).

Read this as

Imagine the image as a topographic map where height equals brightness. An edge is a cliff. We compute how steep the ground is at every point (the gradient magnitude). We keep only the very top of each cliff (non-max suppression). We then keep only the cliffs that are tall enough — but we let small cliffs survive if they connect to tall ones, so the cliff edge doesn’t get broken into pieces.

2.3 Does this solve our problem?

Live Demo

Run `edge_detect.py`, point at the robot, switch to Show Mode 3 (contours).

The robot’s outline appears. So does the floor tile pattern. So do the shadows of cables. So does the edge of the table, the chair leg, the window frame in the background. The image is a forest of contours, and the robot is just one tree in the forest.

Why this fails

Edge detection finds every brightness change. The robot is a brightness change, yes — but so is everything else in the scene. We have moved the problem, not solved it. We can detect “a thing,” but we cannot detect “the robot.”

There is no slider on this pipeline that lets us say “the robot specifically.” The algorithm has no notion of which contour we mean.

What is the missing ingredient? It is information about the *scene*. Edge detection treats every frame in isolation. But we know something about the room: it is not full of moving robots. *The robot is the thing that wasn't there before.*

3 Attempt 3 — “Find What’s New” (robot_outline.py)

Here is the trick that turns the corner. We are going to take two photographs. The first one is of the empty room. The second one is the room with the robot in it. The difference is the robot.

This is called *background subtraction*, and it is one of the most underrated ideas in classical computer vision.

3.1 What robot_outline.py does

The script first asks you to clear the robot out of view. You press **SPACE** to capture the empty scene as the background. Then, on every subsequent frame, it compares the live image to that stored background and isolates whatever is different.

Live Demo

Run `robot_outline.py`. Step out of frame, press **SPACE**. Roll the robot in. A clean green outline appears around it — *and only it*.

This is the moment of the lesson.

3.2 The mathematics of background subtraction

The pipeline is five operations long. Walk through each.

Step 1 — Compute the difference image

Let $B(x, y)$ be the stored background frame (grayscale, blurred), and $I(x, y, t)$ the current frame, similarly processed.

Absolute difference

$$D(x, y) = |I(x, y) - B(x, y)|.$$

D is dark everywhere the scene is unchanged and bright wherever something is different.

Read this as

For every pixel, ask: “how different are you from the empty-room photo?”

Step 2 — Threshold the difference

We turn D into a binary mask:

$$M(x, y) = \begin{cases} 255 & \text{if } D(x, y) > \tau, \\ 0 & \text{otherwise.} \end{cases}$$

Pixels different by more than τ become white; everything else becomes black.

Read this as

“Different enough” becomes white, “not different enough” becomes black. The slider **Diff Threshold** sets the cutoff.

Step 3 — Clean the mask with morphology

The raw binary mask is messy. Small specks of sensor noise leak through, and the robot itself has small dark patches inside it. We clean it with two operations defined on a small structuring element S (a 7×7 ellipse in our defaults).

Erosion shrinks white regions:

$$(I \ominus S)(x, y) = \min_{(i, j) \in S} I(x + i, y + j).$$

Dilation grows white regions:

$$(I \oplus S)(x, y) = \max_{(i, j) \in S} I(x + i, y + j).$$

From these we build closing (fills holes) and opening (removes specks):

Closing and Opening

$$I \bullet S = (I \oplus S) \ominus S \qquad I \circ S = (I \ominus S) \oplus S.$$

We apply Closing then Opening: first heal the robot, then sweep the floor.

Read this as

“Puff up then shrink” fills holes inside the robot. “Shrink then puff up” deletes tiny stray specks of noise outside it. We do them in that order, and the mask becomes a clean, solid blob the shape of the robot.

Step 4 — Find contours

`cv2.findContours` runs the Suzuki–Abe algorithm to trace the boundary of every connected white region in the mask. The output is a list of polygons, each given as an ordered list of (x, y) points.

Read this as

For every white blob in the mask, walk around its edge once and record the corners.

Step 5 — Pick the biggest contour

Among all contours $\mathcal{C}$, we keep only those with area above a minimum threshold $A_{\min}$, then select the single largest:

Largest-contour selection

$$c^* = \arg \max_{c \in \mathcal{C}, A(c) \geq A_{\min}} A(c).$$

The area of a contour with vertices $(x_0, y_0), \dots, (x_{n-1}, y_{n-1})$ is given by the shoelace formula:

$$A = \frac{1}{2} \left| \sum_{k=0}^{n-1} (x_k y_{k+1} - x_{k+1} y_k) \right|.$$

Read this as

The robot is the biggest moving thing in the room. So out of all the white blobs we found, we just keep the largest. That is the robot.

Why this works

Background subtraction works because it incorporates information that edge detection threw away — namely, *the scene the robot is in*. By looking at what’s different from the empty room, we automatically ignore the floor, the cables, the chair, the wall. They were already there. Only the robot is new.

This is also why the method has its limits: if the camera moves or the lighting changes, the stored background no longer matches the live scene, and the whole image lights up as “different.”

We now have what we have wanted from the start: a reliable, frame-by-frame outline of the robot, plus its area in pixels.

4 From an Outline to a Number

The largest contour has an area A , measured in square pixels. The shoelace formula gave us that. So the question becomes: *how do we turn A into a distance?*

To do that we need two things:

1. A reason to believe A is related to d in a specific way. (Otherwise we are guessing.)
2. A small set of measurements that pin down the exact relationship for our setup. (Otherwise we know the shape but not the scale.)

The first comes from the geometry of how a camera works. The second comes from sticking a tape measure on the floor.

4.1 Why distance and area must be related — the pinhole argument

A camera is, to first approximation, a pinhole. Light from a 3D point (X, Y, Z) passes through the pinhole and lands on a sensor at image coordinates

$$x = f \frac{X}{Z}, \quad y = f \frac{Y}{Z},$$

where f is the focal length (in pixels). Both image coordinates scale with $1/Z$.

Now consider an object of real width W and real height H , sitting at distance $d = Z$ from the camera. Its projected width and height in the image are

$$w = \frac{f W}{d}, \quad h = \frac{f H}{d}.$$

Both shrink as $1/d$. So the projected area is

$$A = w \cdot h = \frac{f^2 W H}{d^2}.$$

Solving for distance:

The pinhole distance law

$$d = \frac{k}{\sqrt{A}}, \quad k = f \sqrt{W H}.$$

The constant k packages together the camera's focal length and the robot's physical cross-section. We do not need to measure f , W , or H separately — we only need k , which is one number.

Read this as

When you move twice as far from an object, it looks half as wide AND half as tall in the picture, so its area in the picture shrinks by a factor of four. That is the inverse-square rule, and it means *distance is proportional to one over the square root of area*. The proportionality constant depends on your specific camera and your specific robot. We will measure it.

4.2 Measuring k — the calibration phase

We collect data manually. Place the robot at, say, 5 cm from the camera. Read off the area from `robot_outline.py`. Write down the pair (A_1, d_1) in a spreadsheet. Move the robot to 10 cm. Repeat. Then 15, 20, 25, ... centimetres. Aim for at least ten pairs, covering the whole range of distances you care about.

The spreadsheet ends up with two columns: `area_px` and `distance_cm`.

We now have n pairs $\{(A_i, d_i)\}$ and a model $d = k/\sqrt{A}$ with one unknown. To find k , we minimise the sum of squared errors:

$$\mathcal{L}(k) = \sum_{i=1}^n \left(d_i - \frac{k}{\sqrt{A_i}} \right)^2.$$

Setting $\partial \mathcal{L} / \partial k = 0$ gives a closed-form solution:

Least-squares estimate of k

$$\hat{k} = \frac{\sum_i d_i / \sqrt{A_i}}{\sum_i 1 / A_i}.$$

This is what `fit_distance.py` computes (via `scipy.optimize.curve_fit`).

Read this as

Try lots of values of k . For each one, see how far off the predicted distances are from the measured ones. Add up all those errors (squared, so positive and negative don't cancel). Pick the k that makes the total error smallest. That's our calibrated constant.

In our class calibration the script produced

$$\hat{k} \approx 406.30 \quad (d \text{ in cm, } A \text{ in pixels}^2),$$

with a leave-one-out mean absolute error of about 2 cm.

4.3 Why we fit several models, not just one

`fit_distance.py` actually fits five different models and compares them: the physics model, a free-intercept linearised version, a quadratic polynomial, a cubic polynomial, and a 100-tree random forest. Why?

Key Idea

A model that fits the training points well is not the same as a model that predicts new points well. The right inductive bias beats raw flexibility, especially with little data.

5 Attempt 4 — “Predict the Distance Live” (`robot_distance.py`)

We now have everything we need. The pipeline:

1. A camera frame comes in.
2. Background subtraction gives us the robot’s silhouette.
3. Contour extraction gives us its area A .
4. The physics formula gives us $\hat{d} = k/\sqrt{A}$.

This is what `robot_distance.py` does. It loads $\hat{k}$ from the JSON file that `fit_distance.py` wrote, then runs the `robot_outline.py` pipeline frame by frame, computing the distance from each area and printing it on screen.

Live Demo

Run `robot_distance.py`. Roll the robot to and from the camera. The number on screen should follow the tape measure to within a couple of centimetres.

Two extra ideas are worth mentioning briefly.

5.1 Smoothing the area before computing distance

The raw area jitters from frame to frame because detection is noisy. Plugging the raw A into the formula gives a jittery distance display. We smooth by averaging the last N area readings:

Sliding-window smoothed distance

$$\bar{A}_t = \frac{1}{N} \sum_{i=t-N+1}^t A_i, \quad \hat{d}_t = \frac{\hat{k}}{\sqrt{\bar{A}_t}}.$$

Larger N means a smoother readout but slower response to actual motion. The script exposes this as the Smoothing Window slider.

Read this as

The robot’s area twitches a little even when it’s sitting still — camera noise, lighting flicker. We hide the twitches by averaging the last few readings. Bigger window = smoother numbers but slower to react.

5.2 The extrapolation warning

The model was only validated where we have data — between the smallest and largest areas we saw during calibration. Outside that range, we are guessing.

Extrapolation check

$$\text{in_range}(A) = \mathbf{1}[A_{\min} \leq A \leq A_{\max}].$$

If `False`, the script writes `EXTRAPOLATING` in yellow on the display.

Read this as

The model only knows what it was shown. If you put the robot way closer or way farther than any calibration point, the model is guessing. The screen says so out loud.

Note

A working model that warns about its limits is real engineering. A model that confidently produces wrong numbers outside its valid range is dangerous. Always know where your model is, and isn't, calibrated.

6 What we learned today

We made four attempts. Three of them “failed” in revealing ways:

- **YOLO** failed because pre-trained models can only output classes they were trained on. The robot was not in COCO. *Lesson: “the best ML model” is bounded by its training data.*
- **Edge detection** failed because it has no concept of scene context — it fires on every brightness change, not just ours. *Lesson: classical methods that ignore prior knowledge leave value on the table.*
- **High-capacity regression models** (polynomial, random forest) “failed” the calibration by memorising the noise. *Lesson: more parameters is not more accuracy; the right inductive bias is.*

The fourth attempt worked because it stacked three things in the right order: a classical geometric prior (the empty scene), a classical algorithm to exploit it (background subtraction), and a physics-derived single-parameter model to convert pixels to metres.

Why this works

This is what good engineering looks like. Not every problem needs deep learning. Some problems need a tape measure, six calibration points, and one equation.

Summary of Equations

Concept	Equation
YOLO grid prediction	$(t_x, t_y, t_w, t_h, p_{\text{obj}}, p_{c_1}, \dots, p_{c_{80}})$
Intersection-over-Union	$\text{IoU} = B_1 \cap B_2 / B_1 \cup B_2 $
Image gradient	$\nabla I = (\partial I / \partial x, \partial I / \partial y)$
Gradient magnitude	$M = \sqrt{I_x^2 + I_y^2}$
Absolute difference	$D(x, y) = I(x, y) - B(x, y) $
Binary thresholding	$M = 255$ if $D > \tau$ else 0
Closing	$I \bullet S = (I \oplus S) \ominus S$
Opening	$I \circ S = (I \ominus S) \oplus S$
Largest contour	$c^* = \arg \max_c A(c)$
Shoelace area	$A = \frac{1}{2} \sum (x_k y_{k+1} - x_{k+1} y_k) $
Pinhole projection	$x = fX/Z, y = fY/Z$
Projected area	$A = f^2 WH / d^2$
Distance from area	$d = k / \sqrt{A}, k = f\sqrt{WH}$
Least squares for k	$\hat{k} = \sum_i d_i / \sqrt{A_i} / \sum_i 1 / A_i$
Smoothed distance	$\hat{d}_t = \hat{k} / \sqrt{\bar{A}_t}$
Extrapolation check	$\mathbf{1}[A_{\min} \leq A \leq A_{\max}]$

End of capstone class. Stereo, learned depth, and Kalman filtering follow.